

Mini-Project 5: The Modern Generic Ring Buffer

1. Project Overview

In Mini-Project 1, you built a procedural Ring Buffer tightly coupled to the `int` data type and a hardcoded static array size. In professional C++ firmware, you need buffers that can hold *any* data type (bytes, floats, complex telemetry structs) and allocate their capacity dynamically at runtime without leaking memory.

This project challenges you to convert your procedural C-code into a modern C++ **Template Class** utilizing **Smart Pointers**.

2. Core Objectives

- **Templates:** Use `<typename T>` to allow the buffer to store any data type.
- **Smart Pointers:** Replace the static array with a `std::unique_ptr<T[]>` allocated on the Heap to dynamically set the buffer's capacity at runtime.
- **The Rule of Zero/Five:** Ensure the buffer cleans up its own memory automatically, and explicitly disable copying to prevent duplicate heap allocations.

3. Technical Requirements

- **Language:** C++17
- **Build System:** CMake (Integrated into your workspace as `05_generic_buffer`)
- **Files:** `GenericRingBuffer.h` (Template logic *must* live entirely in the header) and `main.cpp`.

4. Functional Specifications

The Template Class (`GenericRingBuffer.h`)

Create a template `<typename T>` class named `RingBuffer`.

- **Private State:**
 - `std::unique_ptr<T[]> m_data;` (The dynamically allocated array)
 - `size_t m_capacity;`
 - `size_t m_head;`, `size_t m_tail;`, `size_t m_count;`
- **Constructor:** Takes a `size_t capacity`. Uses a Member Initializer List to set the tracking variables to 0, and instantiates the `m_data` array using `std::make_unique<T[]>(capacity)`.
- **Copying Disabled:** Explicitly delete the copy constructor and copy assignment operator (`= delete;`) so no one can accidentally deep-copy this hardware buffer.
- **Methods:**
 - `bool push(T value)` : Adds an element, wraps the head using modulo, increments count. Fails if full.
 - `bool pop(T& out_value)` : Extracts an element, wraps the tail using modulo, decrements count. Fails if empty.

5. Testing & Validation (In `main.cpp`)

Your `main()` function must prove the template generates completely different code for different types:

1. Integer Buffer:

- Instantiate `RingBuffer<int> intBuffer(5);`
- Push `10`, `20`, `30`.
- Pop and print them.

2. String Buffer:

- Instantiate `RingBuffer<std::string> stringBuffer(3);`
- Push `"Engine Boot"`, `"I2C Error"`.
- Pop and print them.

3. Memory Safety Verification:

- You do not need to write a destructor or call `delete`. Verify that the program exits without memory faults, proving the `unique_ptr` handled the cleanup.